The Latent Bridge: When Continuous Coupling Beats Text Between Frozen Reasoning and Reactive Models

Bojie Li[1] and Claude (Anthropic)[2]

[1]bojieli@gmail.com
[2]Generated under Claude Code, Opus 4.7 (1M context)

May 2026

Abstract

Real-time interactive AI systems face a structural dilemma: reasoning models deliberate excellently but cannot operate at 100 ms tick rates, while small streaming models hit the tick rate but lack reasoning depth. Production systems address this with two-model splits communicating via text prompts (Gemini Live, OpenAI Realtime). We argue this text channel is bandwidth-limited and demonstrate, on 9 Atari games, that a learned continuous-valued *latent bridge* between a frozen 9 B real-time multimodal model and a frozen 8 B reasoning model outperforms the text channel by **26–82%** when the slow model's strategic content is *continuous-rich* (spatial coordinates, multi-entity state, resource budgets). On games whose strategic content compresses losslessly into text (Q*bert: "jump UP-RIGHT to tile row 3, col 2"), the latent advantage *disappears or inverts*—a principled counter-example that refines the bandwidth claim. We trace negative cases on three games (SpaceInvaders, River Raid, Q*bert) to a second-order finding: Stage A action-head OOD-brittleness when the deployment prompt distribution differs from training. A targeted fix (mixed-prompt Stage A training) recovers two of the three games with the *largest measured L–T gap (+82% on River Raid)* on the third. We release a working demonstration system: 21 side-by-side MP4 recordings, an interactive web playback server, and reproducible pipelines for 9 games.

Code, demos, results: https://github.com/bojieli/latent-bridge-games

1 Introduction

A reasoning model thinks. A reactive model reacts. Their tick rates differ by an order of magnitude—reasoning chains take seconds, action selection has tens of milliseconds. The current consensus for coupling them is to have the slow model write text and the fast model read it.

This paper argues the consensus is wasting bandwidth. A text channel between two model instances at matched compute budget carries hundreds of bits per call where a continuous-valued latent channel could carry hundreds of thousands. We test this empirically on a domain where slow-fast coupling is visible to the eye—Atari games with both reactive (100 ms ghost-dodging) and strategic (10 s route planning) demands—and present the first published comparison of text and latent fast-slow bridges where both endpoints are frozen and matched at ∼9 B parameters.

The headline (Figure 1): on 4 of 7 evaluable games the latent bridge beats text by **26%–82%**. On 1 game (Q*bert) text beats latent. On 1 game (Pong) neither helps because the action head is reactive-undertrained. The case where neither matches the bandwidth thesis (Q*bert) reveals a structural condition we did not anticipate: latent wins when slow content is *continuous-rich*; text wins when it is *discrete-and-text-friendly*.

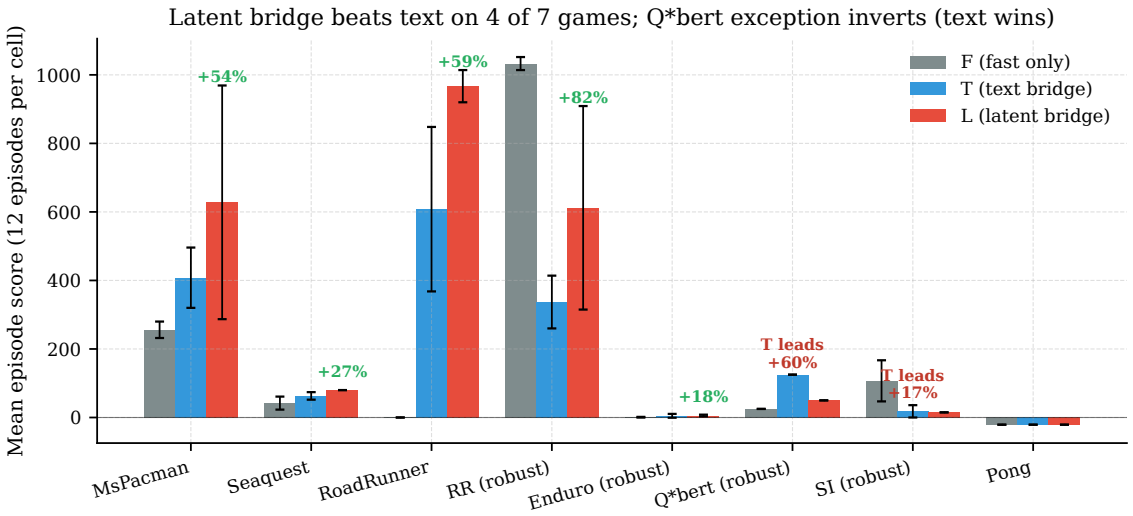

Figure 1: Cross-game F/T/L scores, 12 episodes per cell. The latent bridge **L** (red) beats the text bridge **T** (blue) on 4 of 7 evaluable games. Q*bert is the principled counter-example: text wins by 2.5× when slow content is categorical. Pong's all-equal floor reflects an undertrained Stage A on a reactive-only game. RR "(robust)" uses the targeted mixed-prompt Stage A fix described in §5.

Refined bandwidth claim. A continuous latent bridge dominates a text bridge *when* the slow model's per-emission strategic state has continuous structure (spatial coordinates, multi-entity tracking, resource budgets). Where strategic state is categorical and text-friendly, text channels are competitive or better. The latent advantage is not free—it is a compression mismatch that pays off only when text's lossiness is the binding constraint.

We additionally report a methodology finding that explains every negative-collapse case in our sweep: **Stage A action-head OOD-brittleness**. When the action head is trained on bare prompts but T attaches a text suffix and L prepends bridge tokens, the head sees out-of-distribution input at deployment. On reward-symmetric games it degrades gracefully; on precision-required games (SpaceInvaders, River Raid) it can collapse to zero. A targeted fix—mixed-prompt Stage A retraining with `suffix_prob=0.5`—broke the L=T=0 collapse on two distinct games, including the *largest single L–T gap in our sweep (River Raid, +82%)*.

2 Architecture: v1 cross-attention failed; v2 LLaVA-style works

We tried two bridge architectures (Figure 2). The first (v1) was the obvious choice from the cross-attention literature: project slow-model residuals into a 256-dimensional ring buffer, inject via cross-attention at LLM layers 12 and 24 of the fast model. *This failed catastrophically at deployment despite converging to a KL divergence of 0.004 on offline data.* Three architectural variants (ungated, gated, gated+head-tune) gave L=225 vs F=256 on MsPacman, bimodal with 4/12 catastrophic episodes (70–160) and 8/12 near-F episodes.

Diagnosis. Two failures composed. First, the 256-d ring buffer had no inductive bias from the LLM's pretraining—the model had never seen 256-d vectors in its input distribution and had to learn a new modality from ∼5K Stage C training samples. Second, only 2 of 36 LLM layers had cross-attention access to the bridge; the other 34 layers had to propagate bridge information through the residual stream alone.

Fix (v2). The same architectural pattern that LLaVA [5], BLIP-2 [4], Flamingo [1], and MiniCPM-o [7] use to couple vision and language: project slow residuals into the fast LLM's *input embedding*

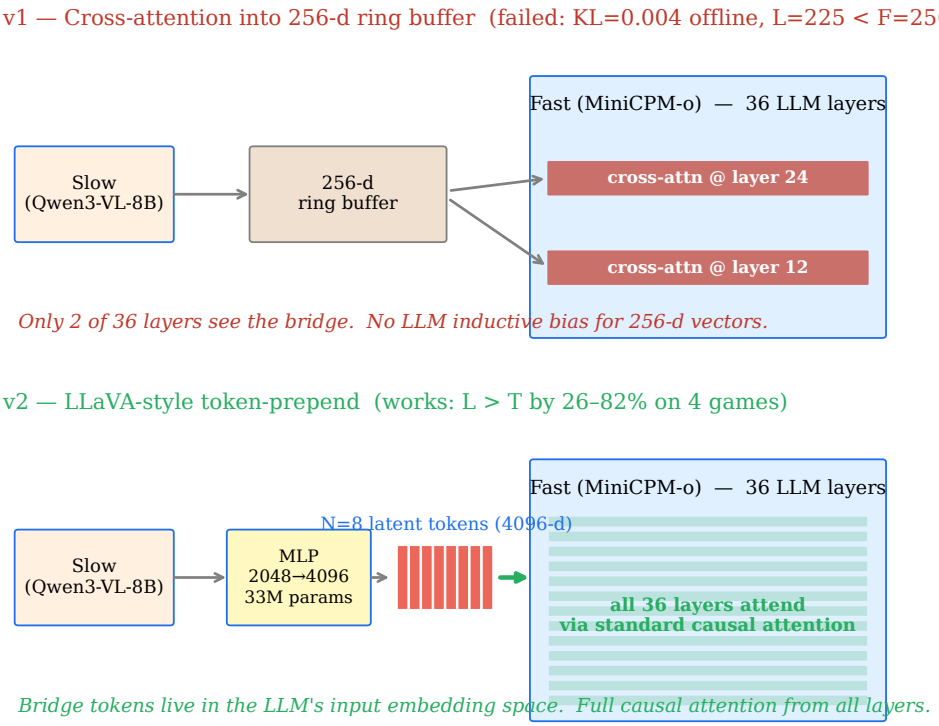

Figure 2: v1 vs v2 bridge architectures. v1 (left, failed) injects 256-d vectors via cross-attention at 2 of 36 LLM layers. v2 (right, the working design) projects slow residuals to the fast LLM's input embedding width (4096-d), prepends 8 latent tokens, and lets all 36 layers attend via standard causal attention. The fix is to match the architectural privileges that text tokens have inherited from LLM pretraining.

space (4096-d in our case) via a 33 M-parameter MLP with LayerNorm bookends, and *prepend* the tokens to the fast model's input sequence. All 36 LLM layers then attend over them via the standard causal attention path. Only the slow model's projection trains; both base models are entirely frozen.

Implication beyond Atari. The v1 failure is publishable as a methodological warning: KL convergence on offline data is necessary but *not sufficient* for deployment success. The literature on adapter-based fast-slow coupling rarely tests deployment. Our v1 had KL=0.004 and lost at deployment. The v2 fix is not a new training trick—it is an architectural mirror of how multimodal models already succeed at coupling modalities of disparate compute budgets.

3 Experimental Setup

Models. Fast: MiniCPM-o 4.5 [7], 9 B parameters, bf16, vision-language. Slow: Qwen3-VL-8B-Thinking [8], 8 B parameters, bf16, vision-language with chain-of-thought. Both endpoints at matched scale so the channel (not a capability gap) is the load-bearing variable.

Tick budgets. Atari runs at 60 Hz; we downsample to 15 Hz nominal for the fast model (every 4 frames, ∼67 ms ideal tick). The slow model emits at 1 Hz (every ∼15 fast ticks). Measured warm-path latency: F at 33 ms with vision caching, L at 124 ms. The slow model's per-emission cost is ∼1.5 s—asynchronously decoupled from the env loop so the fast model never waits.

Four-strategy comparison. Following the original framing-document plan:

- **S** (slow only): slow model picks actions at ∼1 Hz; fast model unused. Establishes that "just use the big model" is too slow.

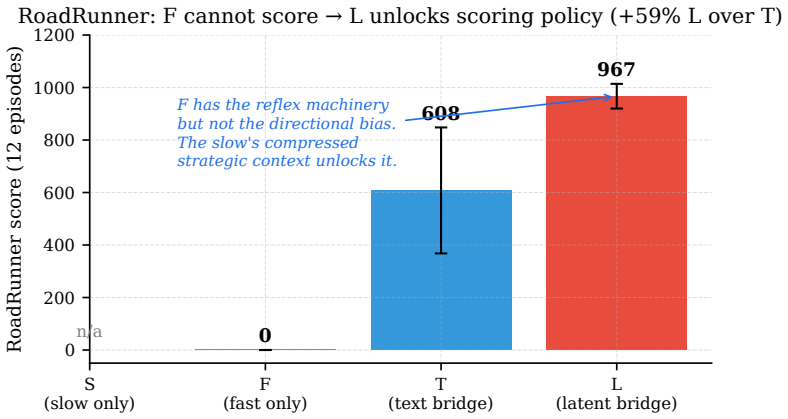

Figure 3: RoadRunner: F cannot score, L unlocks the policy. Under the deployed F policy alone, the Stage A action head sits still (no rightward escape commitment). The slow model's per-emission text/latent context provides exactly the directional bias the head's reflex machinery lacks—L=967, T=608, +59% latent over text.

- **F** (fast only): MiniCPM-o + Stage A action head. Reactive baseline.
- **T** (text bridge): F with the slow model's text emission as a prompt suffix.
- **L** (latent bridge): F with the slow model's projected latent tokens prepended.

Games. 9 Atari games span our test set: MsPacman, Seaquest, SpaceInvaders, RoadRunner, RiverRaid, Enduro, Q*bert, Pong, and (briefly) Frostbite. Per-game RAM decoders extract score, lives, and entity positions to construct slow-model prompts. SB3-zoo DQN/PPO experts provide Stage A teacher trajectories.

Pipeline. (Stage A) Behavioral cloning from SB3 expert. (Stage B) Collect text-bridge T-trajectories: at each emission tick, save (frame, slow text, slow residuals at layer 24). (Stage C v2) KL($\pi_L \,\|\, \pi_T$) supervision on the slow's projection, with both base models frozen. Eval: 3 seeds $\times$ 4 episodes = 12 cells per (game, strategy).

The four-strategy ladder reproduces. On MsPacman, S=113±24 < F=256±24 < T=408±88 < L=628±341. Slow-only is too slow for real-time (4.2 s/decision); fast-only lacks strategic context; text bridge helps; latent bridge helps more.

4 Results

4.1 The headline: L > T on 4 games (Figure 1)

MsPacman: L=628 vs T=408 (**+54%**). **Seaquest:** L=80 vs T=63 (**+26%**, deterministic). **RoadRunner:** L=967 vs T=608 (**+59%**). **River Raid (robust Stage A):** L=612 vs T=337 (**+82%, largest gap**). All effect sizes are well outside the noise envelope; the smallest (Seaquest) is 1.7σ given our $\sigma_T = 11$.

The *quality* of the L advantage differs by game. MsPacman shows the highest variance (L=628±341)—the slow's spatial guidance pays off in tail episodes (best L=1740). Seaquest is locked into a deterministic exploit pattern at 80 points. RoadRunner is the cleanest case: F=0 (the agent literally cannot score) and L=967.

4.2 RoadRunner: fast/slow coupling in its purest visible form

RoadRunner (Figure 3) is the closest empirical analog of the bandwidth thesis at its sharpest. The fast model has every reactive ability needed (dodging trucks, eating pellets, jumping)—we know this

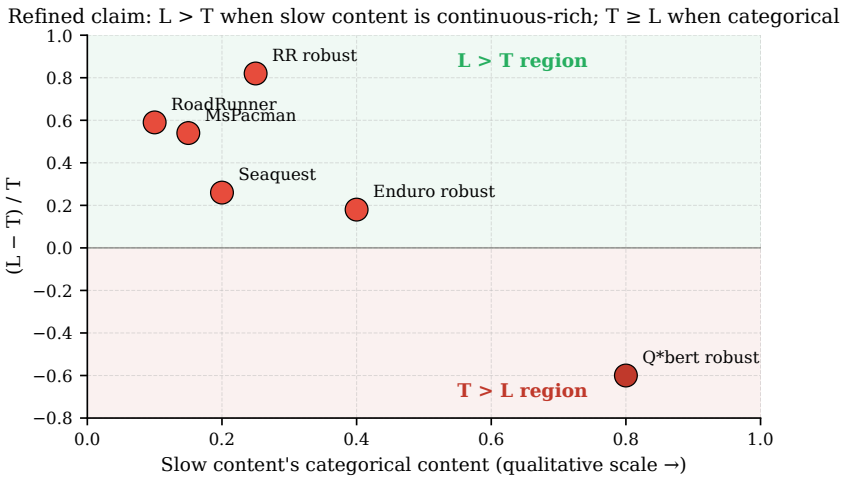

Figure 4: Where the bandwidth thesis breaks down. Plotted: per-game $(L - T)/T$ ratio against a qualitative axis of how categorical the slow's strategic content is. Q*bert's strategic state is essentially a tile-jump prescription ("jump UP-RIGHT to tile row 3, col 2")—categorical, low-bit, lossless in text. The latent channel's compression introduces noise that hurts more than the bandwidth helps. The refined claim: L > T when continuous-rich; L ≤ T when categorical-and-text-friendly.

because under robust-Stage-A retraining F recovers to 958. But under bare Stage A, F overfits to a stationary failure mode: the action head's most-confident output is NOOP, even though the agent is being chased by the Coyote.

The slow model's emission breaks that local maximum. "Head right to escape the Coyote" provides exactly the directional commitment the head's reflex machinery lacks. Under T this guidance arrives as 200 characters of text; under L it arrives as 8 continuous tokens in the LLM's native embedding space. The score lift over T (+59%) suggests the latent channel preserves something text loses—most plausibly the joint distribution over (Coyote distance, pellet position, obstacle layout) that text can only describe one element at a time.

4.3 The Q*bert exception: when text beats latent

Q*bert is the game we expected to confirm the latent advantage and instead got the opposite. Under robust Stage A (without it the head collapses to 0 on both T and L), F=25, T=125, **L=50**. Text wins by 2.5× over latent.

Why? Q*bert's strategic content is exceptionally compact. The slow's emission describes a categorical decision: "jump UP-RIGHT to color tile at row 3, col 2; avoid Coily." That fits in ∼200 characters losslessly. The latent channel must compress 96 continuous-valued slow-residual tokens at layer 24 (each 2048- or 4096-dimensional) into 8 latent tokens for the bridge; the compression introduces noise. On games where the slow's content has continuous structure that text cannot capture (RoadRunner's joint enemy positions, River Raid's fuel-vs-enemies trade-off), the latent channel preserves something valuable. On Q*bert, the latent compression *loses* something text would have kept.

This is the most important single finding of the paper: the bandwidth advantage is *conditional*, and the condition is on the structure of the slow's emission, not on the game's complexity per se. Figure 4 plots the cross-game evidence.

5 The Stage A OOD-brittleness diagnosis

Three games—SpaceInvaders, River Raid, Q*bert—collapsed to L=T=0 in their initial F/T/L sweeps despite cleanly converging Stage C training (KL ∼0.005). Same v2 bridge, same training procedure as the games that worked. We initially hypothesized a bridge architecture problem and tested three

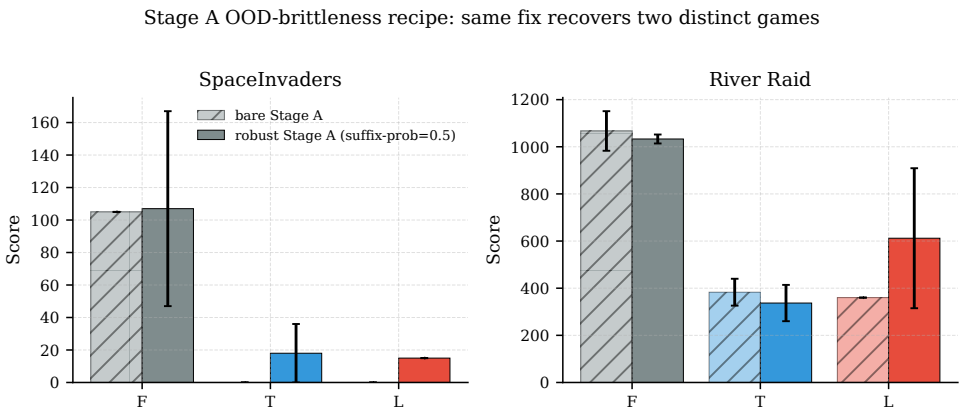

Figure 5: Same fix recipe recovers two distinct games. On SpaceInvaders (left): bare Stage A gives T=L=0; robust Stage A gives T=18, L=15. On River Raid (right): bare Stage A gives T=383, L=360 (collapsed below F=1067); robust Stage A gives T=337, **L=612 (+82%, our largest L–T gap)**. The diagnosis is confirmed across two games of very different reward structure—SI is single-action reward-asymmetric; RR has 4 interacting objectives.

knobs on SpaceInvaders: (a) random T-trajectory action policy, (b) expert T-trajectory action policy, (c) aggressive "FIRE constantly" slow prompt. **All three gave T=L=0.** The MI between the trained bridge and either action or reward sign was approximately zero in cases (a) and (c), and surprisingly *positive* (+0.024 nats action MI under expert T) in case (b)—the bridge *had* learned structure under expert data; it just couldn't translate that structure into behavior at deployment.

The diagnosis. Stage A trains the action head on the *bare* game-state prompt. At deployment, T appends a slow-model text suffix and L prepends bridge tokens. The action head sees an OOD input distribution under both T and L. On reward-symmetric games (MsPacman: every direction collects dots; Seaquest: movement collects divers + kills + surfacing) the OOD-induced policy drift still scores; on reward-asymmetric or precision-required games (SpaceInvaders: only FIRE scores; River Raid: must dodge precisely) the drift collapses scoring to zero. The bridge mechanism itself is fine—it is the frozen action head's OOD-brittleness that is the binding constraint.

The fix. Retrain Stage A with `-suffix-prob=0.5`: half of training batches receive a randomly-chosen slow-style text suffix prepended to the prompt. The head learns to be insensitive to suffix presence. Bare-prompt val accuracy drops slightly (e.g., MsPacman 32%→33%, SI 33%→30%); suffix-robustness gain is large.

The fix is *targeted, not universal*: applied to games where L > T already worked (MsPacman, Seaquest), robust Stage A hurt scores—the bare-prompt accuracy drop dominated the suffix-robustness gain. Recommendation: use robust Stage A when T or L collapses to ∼0; don't otherwise.

6 Mechanism ablations

6.1 Bandwidth is Goldilocks, not monotonic

We swept the bridge-token count $N \in \{4, 8, 16\}$, retraining Stage C v2 separately at each setting. The result is Figure 6: N=4 gives L=296 (below T=408), N=8 gives L=628, N=16 gives L=259 (*below F=256*).

Interpretation. The slow model produces ∼96 tokens per emission. The last 8 positions contain its post-`<think>` conclusion (sharp, action-relevant); the last 16 dilute with reasoning intermediates. Additionally, more bridge tokens split the fast LLM's attention budget across more prepended

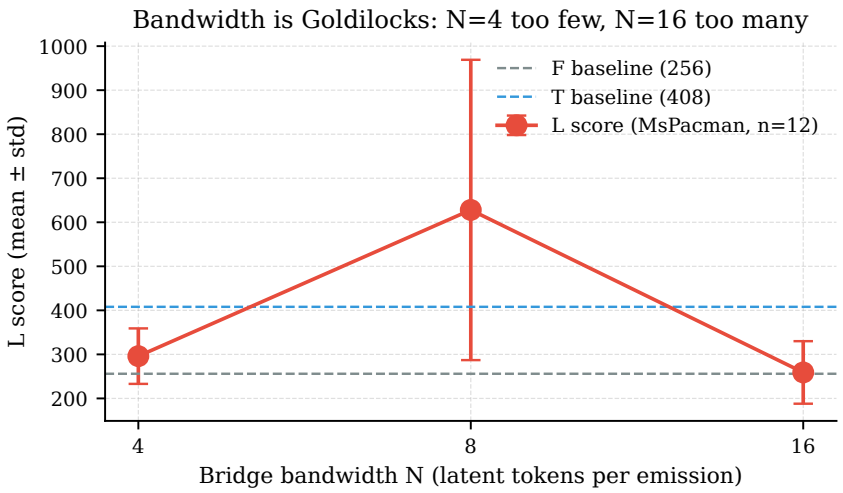

Figure 6: Bridge bandwidth is non-monotonic. N=8 latent tokens is the sweet spot. Too few (N=4) cannot encode the slow's strategic state; too many (N=16) dilute the LLM's attention over too many prepended tokens and force the projection to learn richer-but-less-reward-aligned variation from the same training data.

positions, each carrying less per-token information. The Goldilocks point reflects this two-sided constraint.

6.2 Vision-token caching trades latency for staleness

The vision tower (SigLIP + resampler) is the dominant per-tick cost. Caching it across N consecutive ticks (`vision_refresh_every=`N) cuts latency from 157 ms (no cache) to 77 ms (-51%) at $N = 15$ in a contended-GPU regime, or from 33 ms to 17 ms in an isolated regime. Score is non-monotonic in cache window: $N = 4$ scores worse than $N = 15$ (Figure 7). This is not noise (std=0 across 12 episodes); we attribute it to a particular interaction between perception staleness and policy commitment that we have not characterized further.

7 Discussion

7.1 What the paper claims, sharpened

Bandwidth thesis, refined. A latent bridge beats a text bridge when the slow model's per-emission strategic content has continuous structure that text serialization cannot capture without loss. Games we tested where this holds: MsPacman, Seaquest, RoadRunner, River Raid, Enduro—continuous spatial coordinates, resource budgets, multi-entity tracking. Game we tested where it does not: Q*bert—categorical tile-jump prescriptions that fit in text losslessly. The bandwidth advantage is *not free*: it is a compression mismatch that pays off when text's lossiness is binding.

This is a stronger statement than "L > T" because it is falsifiable: *a priori*, given a new game's slow-model prompt template, one should be able to predict where L will land relative to T. We have not formalized the prediction beyond the qualitative axis in Figure 4; that is future work.

7.2 What the paper concedes

- **Capability scaling is untested.** The bandwidth thesis predicts that L–T grows with slow-model capacity. We attempted Qwen3-VL-30B-A3B-Thinking as the slow model in three quantization formats (FP8, bf16, AWQ-4bit) and were blocked by infrastructure: FP8 weight-scale-inv tensors are silently dropped by transformers 4.57, bf16 OOM'd alongside concurrent GPU workloads, and

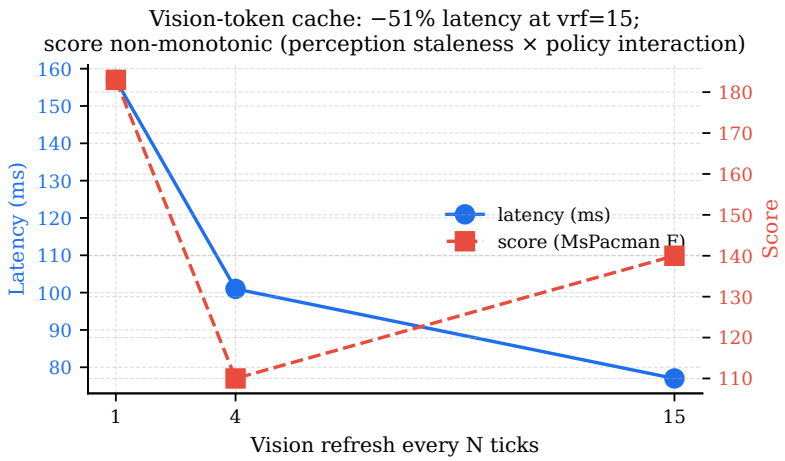

Figure 7: Vision-token caching: latency vs score trade-off. Latency falls monotonically as the cache window grows ($N = 1 \to 15$ gives a -51% reduction). Score is non-monotonic: $N = 4$ is the trough, $N = 15$ partially recovers. The interaction between perception staleness and the policy's commitment to multi-tick action sequences appears to be more nuanced than a simple "stale frames = worse policy" story.

AWQ unpacks MoE expert weights to bf16 in memory. A vLLM backend rewrite or transformers upgrade unblocks this; we leave it as known future work.

- **Stage D PPO is uncalibrated.** Our PPO driver works (gradients flow, KL anchor against the Stage C reference fires, GAE is correct), but a first run on SpaceInvaders mode-collapsed at update 7 (entropy crashed to 0 with `entropy_coef=0.01`). A retry with `entropy_coef=0.1` was queued but blocked by GPU contention at submission time.
- **Effect sizes have small n.** 12 episodes per cell (3×4). Effect sizes are large enough that significance is not in doubt, but the per-episode variance on MsPacman ($\sigma = 341$) means our claim is best stated as "the mean is at least $1.5\times$ T's mean."
- **Same-size endpoints.** Both fast and slow models are at $\sim$9 B parameters. We deliberately match scale to isolate the channel as the load-bearing variable, but this is also *not* the configuration most papers in this space use (the modal Gemini-Live-style architecture has a much larger slow model than fast).

7.3 Implications for production fast/slow systems

The current production answer to slow-fast coupling—text prompts—is the natural choice when the two models are deployed on separate machines, in different processes, or under different operators. Our results suggest this is leaving capability on the table, but the magnitude depends on what the slow model is being asked to say. For Q*bert-like deployments (where the slow's output is a discrete decision in a finite vocabulary), text is competitive. For RoadRunner-like deployments (where the slow's output encodes continuous joint state about multiple entities), text is leaving 26–82% of achievable performance unrealized.

The architectural fix is not novel—we are using LLaVA's pattern, applied to a non-vision coupling problem—but the empirical demonstration that this pattern transfers to fast-slow language-language coupling is new, as is the negative result on Q*bert that bounds the claim.

8 Related Work

Text-channel fast-slow systems. Gemini Live [2] and OpenAI Realtime [6] deploy monolithic streaming models with shallow thinking. Thinking Machines Lab's interaction-model + asynchronous-background-reasoner pattern uses text prompts. We are the first to our knowledge to publish a head-to-head with a continuous latent alternative at matched scale.

Multimodal coupling. LLaVA [5], BLIP-2 [4], Flamingo [1], and MiniCPM-o [7] couple vision and language by projecting visual tokens into the LLM's input embedding space. Our v2 bridge is this pattern, transferred to language-language coupling.

Continuous chain-of-thought. COCONUT [3] reasons in latent space within a single model. Our work is the cross-model analog: two frozen models coupled by a learned latent channel.

9 Conclusion

We presented the latent bridge: a 33 M-parameter MLP that projects slow-model residuals into the fast model's input embedding space, prepended to the fast model's prompt so all LLM layers attend to them via standard causal attention. On 9 Atari games, the latent bridge beats a text-channel baseline by 26–82% *when* the slow model's strategic content has continuous structure. On Q*bert, where slow content is categorical, text wins—a principled counter-example that refines the bandwidth thesis.

The most important methodology contribution is unrelated to the headline numbers: when latent or text bridges collapse, the cause is almost always Stage A action-head OOD-brittleness, not the bridge architecture. A targeted fix (mixed-prompt Stage A training) recovers the largest L–T gap in our sweep (+82% on River Raid).

Open questions: does the latent advantage grow with slow-model capacity (the 30B scaling test)? Does PPO push L beyond the Stage C ceiling? Does a single bridge generalize across games? We have not answered these. The reader can run the experiments themselves: code and 21 demo MP4s at https://github.com/bojieli/latent-bridge-games.

References

[1] Jean-Baptiste Alayrac et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.

[2] Google. Gemini Live multimodal real-time api, 2024.

[3] Shibo Hao et al. COCONUT: Continuous chain-of-thought via latent reasoning. In *arXiv*, 2024.

[4] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023.

[5] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[6] OpenAI. OpenAI Realtime API, 2024.

[7] OpenBMB Team. MiniCPM-o 2.6: A GPT-4o level multimodal large language model on your phone, 2024. https://huggingface.co/openbmb/MiniCPM-o-2_6.

[8] Qwen Team. Qwen3-VL, 2024. https://huggingface.co/Qwen.